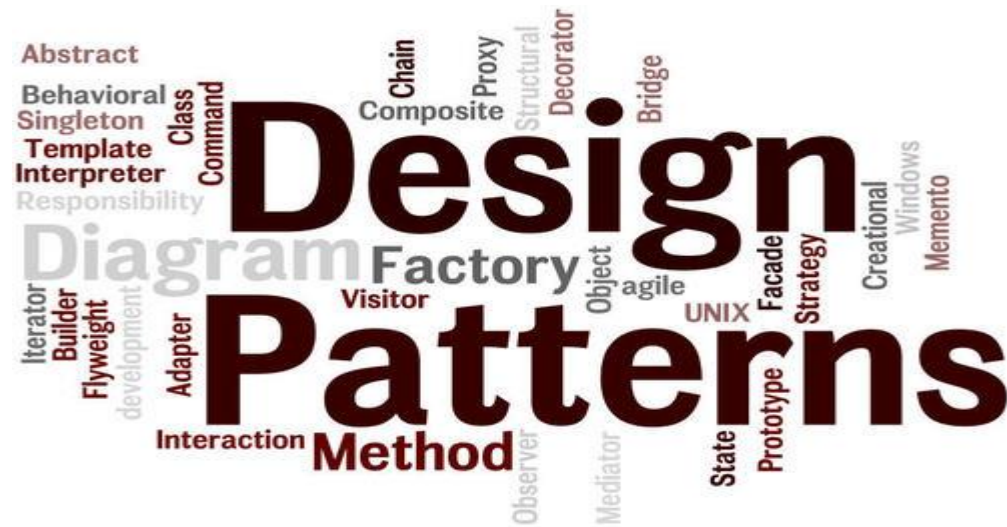




Mẫu thiết kế Model View

ThS. Trần Anh Dũng
dungta@uit.edu.vn

Design pattern

- Là các **giải pháp tổng thể** đã được tối ưu hóa, được **tái sử dụng** cho các **vấn đề phổ biến** trong thiết kế phần mềm mà chúng ta thường gặp phải hàng ngày.
- Là một kỹ thuật trong OOP, không phải là ngôn ngữ cụ thể nào cả.
- Design Pattern giúp bạn giải quyết vấn đề một cách tối ưu nhất, cung cấp cho bạn các giải pháp trong OOP.

Design pattern

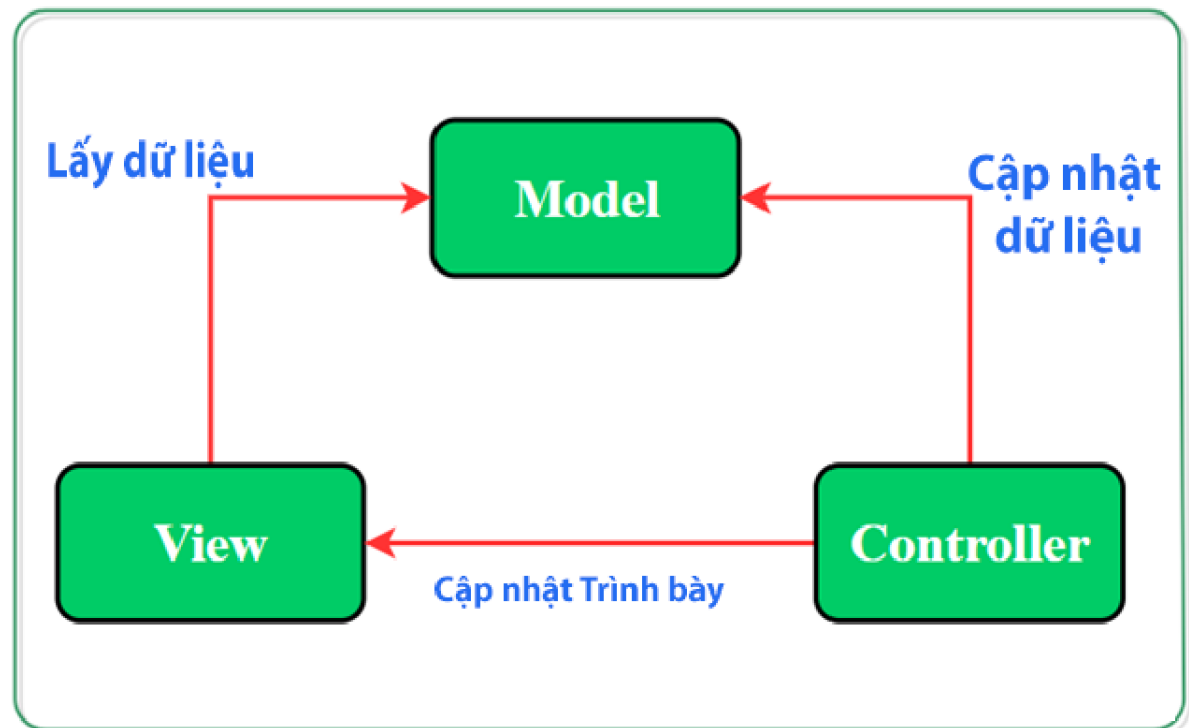
- Design Patterns hỗ trợ developers như thế nào?
 - Tăng tốc độ phát triển phần mềm
 - Code tường minh, dễ dàng team work
 - Tái sử dụng code
 - Hạn chế lỗi tiềm ẩn, dễ dàng nâng cấp

Các mẫu kiến trúc MVC, MVP, MVVM

- Khi phát triển các dự án, các nhà phát triển luôn quan tâm đến việc tổ chức và triển khai các **mô hình kiến trúc** nhằm **tăng hiệu quả**, **giảm thiểu lỗi** và **dễ dàng bảo trì, mở rộng** trong tương lai.
- Ba trong số các mẫu kiến trúc phổ biến nhất hiện nay là Model-View-Controller (**MVC**), Model-View-Presenter (**MVP**) và Model-View-ViewModel (**MVVM**).

Model-View-Controller (MVC)

- **MVC** là một trong những mẫu kiến trúc được biết đến và áp dụng rộng rãi nhất trong ngành công nghiệp phần mềm trước đây.
- Mẫu MVC chia mã thành 3 thành phần.
 - Model
 - View
 - Controller



Model-View-Controller (MVC)

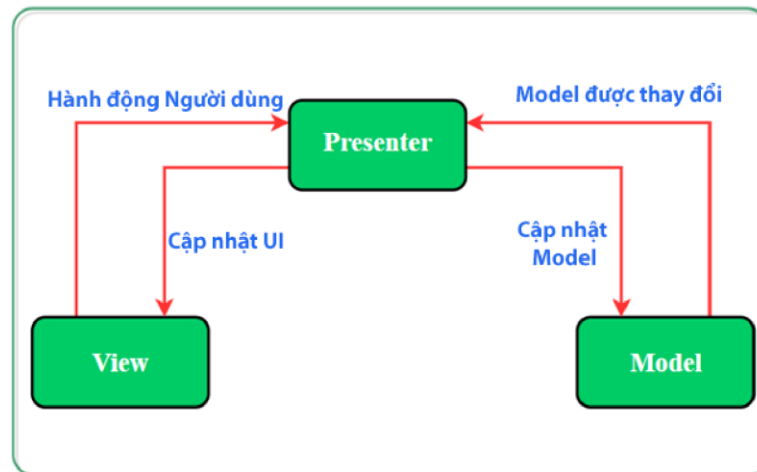
- Ưu điểm:
 - Việc tách biệt các mối quan tâm giúp cải thiện khả năng bảo trì và sử dụng lại mã.
 - Khớp nối lỏng lẻo giữa các bộ phận cho phép sửa đổi và thay thế dễ dàng.
 - Hỗ trợ nhiều biểu diễn trực quan của cùng một dữ liệu.
 - Thúc đẩy phát triển mô-đun và tái sử dụng mã.

Model-View-Controller (MVC)

- **Nhược điểm:**
 - Bộ điều khiển có thể trở thành **nút thắt cổ chai** đối với các **ứng dụng phức tạp** có nhiều tương tác của người dùng.
 - Có thể khó triển khai đối với các ứng dụng có yêu cầu về trạng thái hoặc tương tác phức tạp.

Model-View-Presenter (MVP)

- Mẫu MVP khắc phục những thách thức của MVC và cung cấp một cách dễ dàng để cấu trúc mã dự án. MVP được chấp nhận rộng rãi là vì mẫu kiến trúc này cung cấp tính mô-đun, khả năng kiểm thử và cơ sở mã rõ ràng và dễ bảo trì hơn. MVC sẽ chia thành ba thành phần sau:



Model-View-Presenter (MVP)

- **Model**: Tầng chứa dữ liệu, chịu trách nhiệm xử lý logic miền và giao tiếp với cơ sở dữ liệu và các tầng mạng.
- **View**: tầng giao diện người dùng, cung cấp trực quan hóa dữ liệu và theo dõi hành động của người dùng để thông báo cho Presenter.
- **Presenter**: Lấy dữ liệu từ Model và áp dụng logic giao diện người dùng để quyết định nội dung sẽ hiển thị; Quản lý trạng thái của View và thực hiện các hành động theo thông báo đầu vào của người dùng từ View.

Model-View-Presenter (MVP)

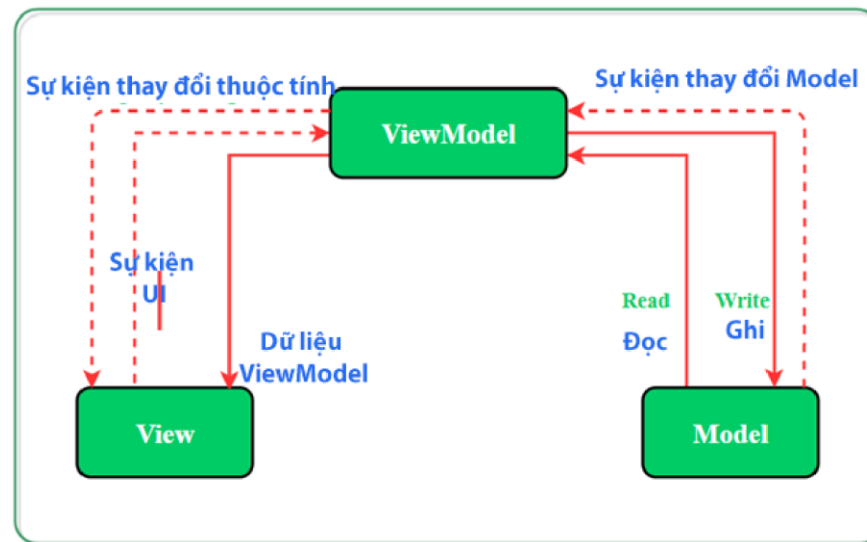
- Ưu điểm:
 - Cải thiện sự phân tách mối quan tâm giữa chế độ xem và mô hình.
 - Người trình bày tạo điều kiện cho khả năng kiểm tra và mô đun hóa tốt hơn.
 - Mỗi thành phần có thể được sửa đổi hoặc thay thế mà không ảnh hưởng đến những thành phần khác.
 - Phù hợp hơn cho các ứng dụng có yêu cầu tương tác hoặc trạng thái phức tạp.

Model-View-Presenter (MVP)

- **Nhược điểm:**
 - Độ phức tạp tăng lên so với MVC truyền thống do người trình bày có thêm trách nhiệm.
 - Có thể dẫn đến một cơ sở mã lớn hơn và cần nhiều mã soạn sẵn hơn.
 - Tiềm năng chi phí liên lạc giữa các thành phần.

Model-View-ViewModel (MVVM)

- Mẫu MVVM có một số tương đồng với mẫu thiết kế MVP do ViewModel đóng vai trò Presenter. Tuy nhiên, nhược điểm của mẫu MVP đã được MVVM giải quyết. MVVM tách logic trình bày dữ liệu (View hay UI (Giao diện người dùng)) khỏi phần logic nghiệp vụ cốt lõi của ứng dụng.



Model-View-ViewModel (MVVM)

- **Model**: chịu trách nhiệm trừu tượng hóa các nguồn dữ liệu. Model và ViewModel phối hợp với nhau để lấy và lưu dữ liệu.
- **View**: Mục đích của lớp này là thông báo cho ViewModel về hành động của người dùng. View quan sát ViewModel và không chứa bất kỳ loại logic ứng dụng nào.
- **ViewModel**: hiển thị các luồng dữ liệu có liên quan đến View. Hơn nữa, phục vụ như một liên kết giữa Model và View.

Mẫu thiết kế MVC, MVP và MVVM

MVC(MODEL VIEW CONTROLLER)	MVP(MODEL VIEW PRESENTER)	MVVM(MODEL VIEW VIEWMODEL)
Một trong những kiến trúc phần mềm lâu đời nhất	Được phát triển như là phiên bản thứ hai của kiến trúc phần mềm, cải tiến từ MVC.	Mẫu kiến trúc được ngành công nghiệp công nhận cho các ứng dụng.
Giao diện người dùng (View) và cơ chế truy cập dữ liệu (Model) được kết hợp chặt chẽ với nhau.	Giải quyết vấn đề phụ thuộc View bằng cách sử dụng Presenter (Người thuyết trình) làm kênh liên lạc giữa Model và View.	Mẫu kiến trúc này là hướng sự kiện nhiều hơn vì sử dụng liên kết dữ liệu và do đó giúp dễ dàng tách logic nghiệp vụ cốt lõi khỏi View.

Mẫu thiết kế MVC, MVP và MVVM

MVC(MODEL VIEW CONTROLLER)	MVP(MODEL VIEW PRESENTER)	MVVM(MODEL VIEW VIEWMODEL)
UI(View) and data-access mechanism(Model) are tightly coupled.	It resolves the problem of having a dependent View by using Presenter as a communication channel between Model and View.	This architecture pattern is more event-driven as it uses data binding and thus makes easy separation of core business logic from the View.
Controller và View tồn tại với mỗi quan hệ một-nhiều (One-to-many). Một Controller có thể chọn một View khác dựa trên hoạt động được yêu cầu.	Mối quan hệ một-một (one-to-one) tồn tại giữa Presenter và View khi lớp Presenter quản lý một View tại một thời điểm.	Nhiều View có thể được ánh xạ với một ViewModel duy nhất và do đó, mối quan hệ một-nhiều (one-to-many) tồn tại giữa View và ViewModel.

Mẫu thiết kế MVC, MVP và MVVM

MVC(MODEL VIEW CONTROLLER)	MVP(MODEL VIEW PRESENTER)	MVVM(MODEL VIEW VIEWMODEL)
View không có kiến thức về Controller.	View có tham chiếu đến Presenter	View tham chiếu đến ViewModel
Khó thực hiện thay đổi và sửa đổi các tính năng của ứng dụng vì các tầng mã được liên kết chặt chẽ với nhau.	Các tầng mã được liên kết lỏng lẻo và do đó dễ dàng thực hiện các sửa đổi/thay đổi trong mã ứng dụng.	Dễ dàng thực hiện các thay đổi trong ứng dụng. Tuy nhiên, nếu logic liên kết dữ liệu quá phức tạp, việc gỡ lỗi ứng dụng sẽ khó hơn một chút.

Mẫu thiết kế MVC, MVP và MVVM

MVC(MODEL VIEW CONTROLLER)	MVP(MODEL VIEW PRESENTER)	MVVM(MODEL VIEW VIEWMODEL)
Đầu vào người dùng được xử lý bởi Controller	View là điểm vào của Ứng dụng	View nhận đầu vào từ người dùng và đóng vai trò là điểm vào của ứng dụng.
Chỉ lý tưởng cho các dự án quy mô nhỏ.	Lý tưởng cho các ứng dụng đơn giản và phức tạp.	Không lý tưởng cho các dự án quy mô nhỏ.
Hỗ trợ hạn chế cho kiểm thử Đơn vị.	Dễ dàng thực hiện kiểm thử Đơn vị nhưng sự liên kết chặt chẽ giữa View và Presenter có thể gây khó khăn đôi chút.	Khả năng kiểm thử đơn vị cao nhất trong kiến trúc.

Mẫu thiết kế MVC, MVP và MVVM

- Sự khác biệt chính giữa MVP và MVC là Presenter trong MVP có vai trò tích cực hơn trong giao tiếp giữa Model và View và chịu trách nhiệm kiểm soát luồng dữ liệu giữa hai bên.
- Trong mẫu MVVM, Model đại diện cho dữ liệu và logic của ứng dụng, View đại diện cho giao diện người dùng của ứng dụng và “ViewModel” là một lớp nằm giữa Model và View và chịu trách nhiệm hiển thị dữ liệu và logic của Model sang View theo cách dễ làm việc hơn.

Mẫu thiết kế MVC, MVP và MVVM

- Nhìn chung, sự khác biệt chính giữa các mẫu này là vai trò của thành phần hòa giải. Cả MVC và MVP đều liên quan đến Controller hay Presenter đóng vai trò trung gian giữa Model và View, trong khi MVVM liên quan đến ViewModel đóng vai trò trung gian giữa Model và View.
- MVC là mẫu đơn giản nhất trong số các mẫu này, trong khi MVP và MVVM linh hoạt hơn và cho phép phân tách rõ ràng hơn các mối quan tâm giữa các layers khác nhau của ứng dụng.

Q & A

